Innominatum CONSULTORIA

GitHub Actions e Integração Contínua

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Objetivo

Compreender o que é a Integração Contínua e porquê automatizar a verificação do código, dominar os conceitos do GitHub Actions, e construir um pipeline CI funcional para um projeto Angular com lint, build e cache de dependências.

Objetivos Específicos

Ao final da sessão, os formandos deverão ser capazes de:

- Explicar o que é CI e o problema que resolve;
- Identificar os conceitos-chave do GitHub Actions: workflow, trigger, job, step e runner;
- Configurar secrets, cache de dependências e quality gates que bloqueiam merges com erros.

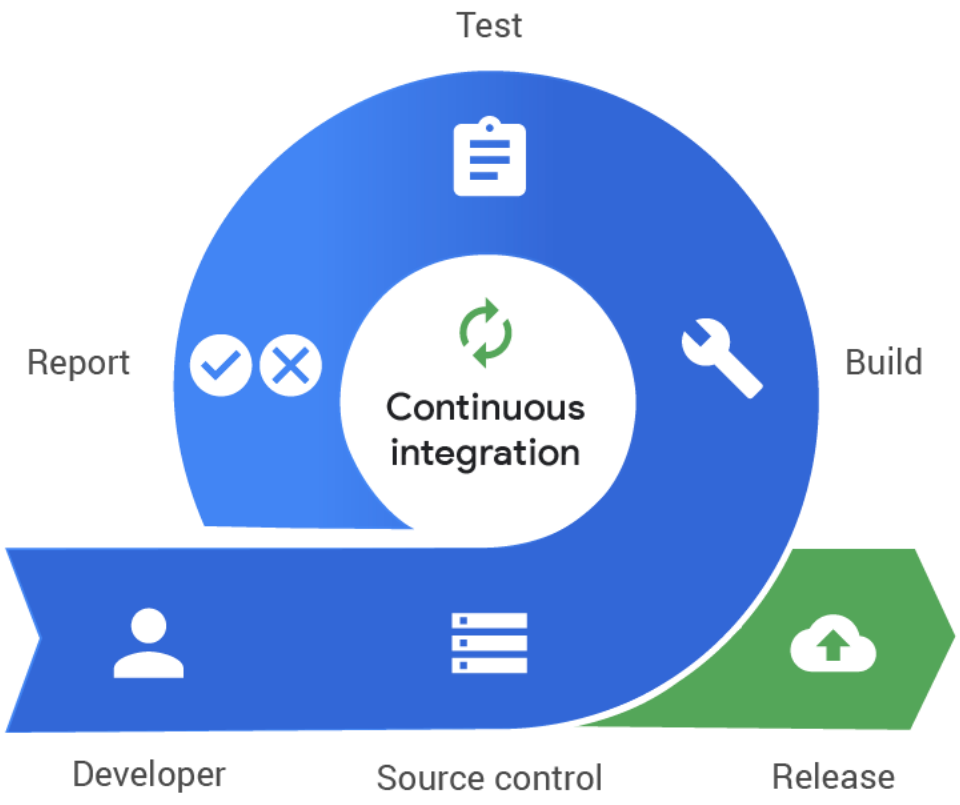
Innominatum CONSULTORIA

Integração Contínua

Automatizar a verificação do código em cada commit

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Integração Contínua



Definição

- **A integração contínua** é uma prática de desenvolvimento de software **DevOps** em que os programadores integram regularmente as suas alterações de código num repositório central, após o que são executadas **compilações (builds)** e **testes automatizados**.

Definição

- A integração contínua refere-se, na maioria das vezes, à fase de **compilação** ou **integração** do processo de lançamento de software e envolve tanto uma **componente de automatização** (por exemplo, um serviço de **CI** ou de **build**) como uma **componente cultural** (por exemplo, aprender a integrar frequentemente).

Definição

- Os principais objetivos da integração contínua são **detetar e corrigir erros mais rapidamente**, **melhorar a qualidade do software** e **reduzir o tempo necessário para validar e disponibilizar novas atualizações de software**

Porque é que é necessário

No passado, os programadores de uma equipa podiam trabalhar de forma isolada durante um período prolongado e só integravam as suas alterações na **branch principal (main)** quando o trabalho estivesse concluído.

Porque é que é necessário

Isto tornava a **integração de alterações de código** difícil e demorada, e também fazia com que os **erros (bugs)** se acumulassem durante muito tempo sem serem corrigidos.

Estes fatores dificultavam a capacidade de **entregar atualizações aos clientes de forma rápida**.

Como é que funciona?

Com a integração contínua, os programadores fazem **commits frequentes** para um **repositório partilhado**, utilizando um sistema de controlo de versões como o **Git**.

Como é que funciona?

Antes de cada commit, os programadores podem optar por executar **testes unitários locais** no seu código como uma camada extra de verificação antes da integração.

Como é que funciona?

Um serviço de **integração contínua** compila automaticamente o código e executa **testes unitários** sobre as novas alterações, permitindo **detetar imediatamente quaisquer erros**.

O Problema sem CI

- O clássico cenário: o código funciona na máquina de cada um, mas quando é integrado no servidor partilhado, alguma coisa quebra. Ninguém sabe bem onde nem quando.

O Problema sem CI

- O clássico cenário: o código funciona na máquina de cada um, mas quando é integrado no servidor partilhado, alguma coisa quebra. Ninguém sabe bem onde nem quando.
- Sem verificação automática, lint e testes são ignorados sob pressão. A base de código degrada-se silenciosamente. A produção torna-se imprevisível.

O Problema sem CI

CI resolve isto com uma regra simples: cada push despoleta uma verificação automática. Se o código não passa, não entra. A qualidade torna-se um processo, não uma opção.

O que o CI Faz na Prática

- Em cada push ou pull request, o sistema CI clona o repositório numa máquina limpa, instala as dependências e executa uma sequência de verificações definidas pela equipa.

O resultado é visível no GitHub:

- verde significa que tudo passou
- vermelho bloqueia o merge do PR.

A equipa têm feedback imediato.

Verificações típicas: lint (o código respeita as regras de estilo?), testes unitários (a lógica está correta?), build (o projeto compila sem erros?).

GitHub Actions

CI/CD integrado no repositório

GitHub Actions

GitHub Actions é a plataforma de automação do GitHub. Permite definir workflows que correm em resposta a eventos no repositório, como pushes, pull requests ou schedules.

Conceitos-Chave

- **Workflow**: ficheiro YAML guardado em .github/workflows/. Define o que corre, quando e em que condições. Um repositório pode ter vários workflows.
- **Trigger**: o evento que despola o workflow. Exemplos: push, pull_request, workflow_dispatch (manual) ou schedule (cron).

Conceitos-Chave

- **Job**: unidade de trabalho dentro do workflow. Cada job corre numa máquina própria. Jobs podem ser paralelos ou sequenciais com needs.
- **Step**: cada passo dentro de um job. Pode ser um comando shell (run:) ou uma action reutilizável do Marketplace (uses:).
- **Runner**: a máquina virtual que executa o job (ubuntu-latest é o padrão).

Triggers, Jobs, Steps e Runners

A hierarquia de execução no GitHub Actions:

- **Trigger (on:):** define quando o workflow arranca. **pull_request** filtra por branch de destino. **push** filtra por branch de origem. **workflow_dispatch** permite execução manual.
- **Job (jobs:):** cada job é uma máquina virtual independente. Por omissão correm em paralelo. **needs:** [job-anterior] torna-os sequenciais.

Triggers, Jobs, Steps e Runners

- **Step (steps:):** dentro de cada job, os steps correm em sequência na mesma máquina. Partilham o mesmo workspace e variáveis de ambiente.
- **Action (uses:):** código reutilizável do Marketplace. actions/checkout clona o repo. actions/setup-node instala o Node.js. actions/cache guarda dependências.
- **Runner (runs-on:):** ubuntu-latest é gratuito e suficiente para a maioria dos casos. Windows e macOS também estão disponíveis mas consomem mais minutos.

Innominatum CONSULTORIA

Anatomia de um Workflow

O ficheiro ci.yml passo a passo

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Estrutura do Ficheiro YAML

- O ficheiro .github/workflows/ci.yml segue uma estrutura **hierarquica fixa**.
- O YAML é sensível a indentação: dois espaços por nível, **nunca tabs**.

Estrutura do Ficheiro YAML

- **name:** dá um nome ao workflow visível no separador Actions do GitHub.
- **on:** define os triggers.
- **jobs:** contém todos os jobs do workflow.

Estrutura do Ficheiro YAML

Dentro de cada job:

- **runs-on** define o runner,
- **steps** lista os passos em ordem. Cada step tem name (descrição visível) e **uses** ou **run** (o que executa).

Estrutura do Ficheiro YAML

- Variáveis de ambiente podem ser declaradas ao nível do workflow (**env:**), do job ou do step.
- O scope é hierárquico: o mais específico sobrescreve o mais geral.

Checkout e Setup Node

- O **primeiro step** de qualquer pipeline **CI Angular** é sempre **actions/checkout@v4**. Sem este step, a máquina virtual não tem acesso ao código do repositório.
- O **segundo step** é **actions/setup-node@v4** com a versão do Node especificada. **Garante que todos os builds usam exatamente a mesma versão**, independentemente do runner.

Checkout e Setup Node

- A boa prática é definir a versão do Node no ficheiro .nvmrc ou no package.json (engines.node) e referenciar esse valor no workflow com node-version-file: .nvmrc.
- De seguida, **npm ci** instala as dependências de forma determinística a partir do **package-lock.json**. É mais rápido e mais seguro que npm install em pipelines CI.

Pipeline CI para Angular

checkout → setup-node → npm ci → lint → build

Os Passos do Pipeline

O pipeline CI para um projeto Angular segue esta sequência:

- **Step 1** — Checkout: actions/checkout@v4. Clona o repositório no runner. Sem isto, não há código para trabalhar.
- **Step 2** — Setup Node: actions/setup-node@v4 com **node-version: "20".** Garante a versão correta do Node e do npm.

Os Passos do Pipeline

- **Step 3 — Instalar: npm ci**. Instala dependências de forma determinística. Falha se o package-lock.json estiver desatualizado.
- **Step 4 — Lint: npm run lint**. Verifica regras de estilo e qualidade do código. **Se falhar**, o pipeline pára aqui e o **PR não pode ser fundido**.
- **Step 5 — Build: npm run build**. Compila o projeto Angular. Confirma que o código compilado é válido. O artefacto não é publicado nesta fase.

Cache de Dependências

O **npm ci** numa instalação limpa demora entre 30 segundos e 2 minutos dependendo do tamanho do projeto. Multiplicado por dezenas de execuções diárias, é tempo considerável.

Cache de Dependências

A action **actions/cache@v4** guarda a pasta **~/.npm** entre execuções. Na segunda vez que o pipeline corre com o mesmo **package-lock.json**, **restaura a cache e salta o download**.

Cache de Dependências

A **chave da cache** é calculada a partir do **hash** do **package-lock.json**. Se o ficheiro mudar, a cache é invalidada automaticamente e recriada com as novas dependências.

Ganho típico: pipeline sem cache em 90 segundos, com cache em 20 segundos. Em projetos grandes a diferença pode ser de minutos. Os minutos de Actions são limitados no plano gratuito.

Innominatum CONSULTORIA

Secrets e Quality Gates

Proteger credenciais e garantir padrões de qualidade

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Secrets no GitHub

Secrets são variáveis de ambiente **cifradas** guardadas no GitHub, nunca visíveis nos logs nem no código. Acede-se a elas no **workflow** com **${{ secrets.NOME_DO_SECRET }}**.

Configurar em **Settings** → **Secrets and variables** → **Actions do repositório**. Podem ser ao nível do repositório ou do ambiente (*environment*), com controlo de acesso separado.

Secrets no GitHub

Casos de uso: SUPABASE_URL e SUPABASE_ANON_KEY para o build Angular, tokens de deploy para Vercel ou Netlify, credenciais de registry para publicar imagens Docker no GHCR.

Boas práticas: usar o princípio do mínimo privilégio (só as permissões necessárias), rodar tokens periodicamente, e nunca usar a service_role key do Supabase no frontend ou no CI.

Quality Gates

O que deve bloquear um pull request:

- **Lint falha**: código que não respeita as regras de estilo não entra na base de código. Evita discussões de formato em **code review**.
- **Build falha**: código que não compila não entra. Tópico inegociável. Um build vermelho deve ser a prioridade máxima da equipa.
- **Testes falham**: unitários ou de integração. Quando adicionados ao pipeline, qualquer regressão bloqueia automaticamente o merge.

Quality Gates

- **Branch protection rules no GitHub**: **Settings** → **Branches** → **Add rule**. Marcar "Require status checks to pass". Selecionar o job do **CI** como **check obrigatório**.
- Com branch protection ativo, nem o administrador do repositório pode fazer merge sem o CI verde. A qualidade deixa de depender da disciplina individual.

Innominatum CONSULTORIA

Badge e Branch Protection

Tornar o estado do CI visível para toda a equipa

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL

Fluxo Completo: do Commit ao Merge

O fluxo CI com GitHub Actions, uma vez configurado, **corre automaticamente** em cada interacção com o repositório **sem qualquer intervenção manual**.

- **Passo 1**: o programador cria uma **feature branch**, faz **commit** e **abre um pull request para main**. O **trigger pull_request** despola o **workflow ci.yml**.
- **Passo 2**: o GitHub Actions cria um runner ubuntu-latest limpo, executa os steps na ordem definida: **checkout**, **setup-node**, **npm ci** (com cache), **lint**, **build**.

Fluxo Completo: do Commit ao Merge

- **Passo 3**: se todos os **steps passam**, o **check fica verde no PR**. Se **algum falha**, o **check fica vermelho**, o **merge fica bloqueado**, e o autor recebe notificação com o log do erro.
- **Passo 4**: com o **CI verde** e **aprovacão de pelo menos um revisor** (se configurado), o **PR é integrado na main**. O **badge** no README **reflete em tempo real** o **estado do último build**.
- **Badge**: em **Actions** → **workflow** → **Create status badge**. Cola o markdown no README. Qualquer pessoa que visite o repositório vê imediatamente se o projeto está saudável.

Innominatum CONSULTORIA

GitHub Actions e Integração Contínua

ServiceNow - Deloitte

Prof. Tiago Rodrigues

IPCA POLITÉCNICO DO CÁVADO E DO AVE
INSTITUTO DO EMPREGO E FORMAÇÃO PROFISSIONAL
PRR Plano de Recuperação e Resiliência
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
IAPMEI Parcerias para o Crescimento
PORTUGAL DIGITAL